ECE 7670
Program 8
Nathan Walker

November 14, 2025

---

1. create a class RSenc which implements a Reed-Solomon encoder fro primitive, narrow-sense codes. verify that the encoder works.

2. create a class RSddec that implements a decoder for a primitive, narrow-sense code.

3. test your decoder by decoding 10,000 patterns of up to 3 errors for the (255,249) code over the field GF(2^8).

```
$ ./testRS.exe
All errors correted!
encoding and decoding worked!
```

Figure 1: terminal output of testRS.exe that shows correct functionality.

4. write the decoder using euclideans algorithm instead.

5. with rsencode and rsdecode write programs that will protect your files on your computer for a (255,255-2t) RS code.

```
1  //  Program: rsdecode.cc --- decode a data file
2  //  Todd K. Moon
3  //
4  // rsencode [-t t] infile outfile
5  //
6  // [-t t] -- specify random error correction capability.
7  //           default = 3
8
9  // In encodding, the data are read with the first byte in the block
10 // corresponding to the constant coefficient of m(x).  In order to
11 // handle the last block of the file correctly, the last block is
12 // written out with its length.  Thus, to decode correctly, the last
13 // block should use the length and write only the correct number of
   bytes
14
15 #include "RSdec.h"
16 extern "C" {
17 #include <stdio.h>              // fopen, fread, fwrite functions
18 #include <string.h>             // strcmp function
```

```
#include <stdlib.h>             // atoi function
}
#include "polynomialT.cc"
// create instantiations of the polynomial class of type GFNUM2m
template class polynomialT<GFNUM2m>;
template class polytemp<GFNUM2m>;

int main(int argc, char *argv[])
{
   GFNUM2m::initgf(8,0x11D);  //   1 0001 1101   x^8 + x^4 + x^3 + x^2 + 1
   GFNUM2m::setouttype(vector);
   int i;
   int t = 3;                   // default error correction capability
   FILE *fin, *fout;
   if(argc < 3) {
      cout << "Usage: " << argv[0] << "[-t t] infile outfile\n";
      exit(-1);
   }
   i = 1;
   if(!strcmp(argv[i],"-t")) { // if t is specified
      ++i;
      t = atoi(argv[i++]);
   }
   int n = 255;                 // coded block length
   int k = 255-2*t;             // input block length
   RSdec decoder(t,n);          // build the encoder object
   unsigned char r[n];          // data space to read in
   unsigned char  dec[n];       // decoded data

   if((fin = fopen(argv[i++],"rb")) ==NULL) {// open input file to read
      cout << "Cannot open file to read\n";
      exit(-1);
   }
   if((fout = fopen(argv[i],"wb")) == NULL) {  // open output file to write
      cout << "Cannot open file to write\n";
      exit(-1);
   }
   int nread, nwritten;
   int prev = 0;
   while((nread = fread(r, 1, n, fin))) {
      // not the last block --- write out previous message
      if(prev) {
         fwrite((void *)(dec+n-k),1,k,fout);
      }
      decoder.decode(r,dec);        // decode the data
      prev = 1;
   }
```

```
66     // now write out the last block
67     // find the last nonzero element
68     int blocklen=0;
69     for(i = n-1; i >= 0; i--) {
70        if(dec[i]) {
71           if(i==n-1) { // if the block is full
72              if(dec[i]==n-1) blocklen = dec[i];
73              else blocklen = n-(n-k);
74           }
75           else {
76              blocklen = dec[i];
77           }
78           break;
79        }
80     }
81     cout << "last block: blocklength = " << blocklen << endl;
82     fwrite((void *)(dec+n-k),1,blocklen,fout);
83
84     fclose(fin);
85     fclose(fout);
86  }
87
88  /*
89  Local Variables:
90  compile-command: "g++ -o rsdecode -g rsdecode.cc RSdec.cc GFNUM2m.cc
       ChienSearch.cc"
91  End:
92  */
```

Listing 1: RSdecode.cc

```
1  // RSdec.cc -- a general Reed-Solomon decoder
2  // Todd K. Moon
3  // Copyright 2004 by Todd K. Moon
4  // Permission is granted to use this program/data
5  // for educational/research only
6
7  #include "RSdec.h"
8
9  #include "berlmass.cc"
10
11 //#include "gcdpoly.cc"
12 // #define BMALG                    // define this to use BM alg.
       for decoding
13 // otherwise, Euclidean algorithm is used for decoding
14
15
16 // construtor
17 RSdec::RSdec(int t_in, int n_in, GFNUM2m A1_in)
18    :Searcher(t_in)
19 {
```

```
20      t = t_in;
21      t2 = 2*t_in;
22      n = n_in;
23      A1 = A1_in;
24      s = new GFNUM2m[2*t];
25      Lambda = new GFNUM2m[t+1];
26      spoly.setc(t2,1);  // allocate space for this
27      Lambdap.setc(t,1); // allocate space for this
28  }
29
30  int RSdec::decode(unsigned char *r, unsigned char *dec)
31  {
32      int i;
33      GFNUM2m rgf[n];
34      GFNUM2m decgf[n];
35      for(i = 0; i < n; i++) {
36          rgf[i] = r[i];
37      }
38      int ret = decode(rgf,decgf);
39      for(i = 0; i < n; i++) {
40          dec[i] = decgf[i].getv();
41      }
42      return ret;
43  }
44
45  int
46  RSdec::decode(GFNUM2m *r, GFNUM2m *dec)
47  {
48      int i,j;
49      int errloc;
50      GFNUM2m p,dp2;              // fast computation of formal
            derivative
51      GFNUM2m x;
52      GFNUM2m *rs;                  // array of roots
53       int no_error = 1;
54
55      // Step 1: evaluate the syndromes
56       // cout << "Syndromes are " << endl;
57      for(i = 0; i < 2*t; i++){
58            GFNUM2m sum = GFNUM2m(0);
59            GFNUM2m evaluating_term_i = A^(i+1);
60            //cout << "i = " << i << " term is " << evaluating_term <<
                 endl;
61            for(j = 0; j < n;j++){
62                sum += r[j]*(evaluating_term_i^j);
63                //cout << "i = " << i << " j = " << j << "
                      evaluating_term * r[j] = " << evaluating_term * r[j]
                      << endl;
64            }
65            s[i] = sum;
66            // cout << s[i] << " ";
```

```
    }
    // cout << endl;
    // cout << 1 + A + (A^2) << endl;
    // cout << 1 + (A^2) + (A^4) << endl;
    // cout << 1 + (A^3) + (A^6) << endl;
    // cout << 1 + (A^4) + (A^8) << endl;
    // cout << 1 + (A^5) + (A^10) << endl;
    // cout << 1 + (A^6) + (A^12) << endl;

   // Step 2: Determine the error locator polynomial
   polynomialT<GFNUM2m> error_locator_poly = berlmass(s,t2);
    // cout << "error_locator_poly = " << error_locator_poly << endl
       ;
    if(error_locator_poly.getdegree() == 0){
        for(i = 0; i < n; i++){
            dec[i] = r[i];
        }
        return 1;
    }
    // cout << "Lambda = " ;
    for(i = 0; i < t+1; i++){
        if(i <= error_locator_poly.getdegree())
            Lambda[i] = error_locator_poly[i];
        else
            Lambda[i] = GFNUM2m(0);

        // cout << Lambda[i] << " " ;
        //error_locator_poly[i] = GFNUM2m(1);
    }
    // cout << endl;
   // Step 3: Find the roots of the error locator polynomial
   int nroots;
    if(error_locator_poly.getdegree()>0){
        rs = Searcher.Search(Lambda, error_locator_poly.getdegree(),
            nroots);
    }
    // cout << "nroots = " << nroots << " roots of the polynomial =
       " << rs[0] << " " << rs[1] << " " << rs[2] << endl;

   // Step 4: Find error values: Not necessary for binary BCH codes
    polynomialT<GFNUM2m> syn_poly(t2-1,s);
    polynomialT<GFNUM2m> prod = syn_poly*error_locator_poly;
    polynomialT<GFNUM2m> xt2(GFNUM2m(0));
    xt2.setc(t2,GFNUM2m(1));
    Omega = prod % xt2;
    // cout << " Omega = " << Omega << endl;

   //determine a function to compute the formal derivative of a
      polynomial.
    polynomialT<GFNUM2m> dx_coeff;
    dx_coeff.setc(error_locator_poly.getdegree()-1, GFNUM2m(0));
```

```
    for(i = 1; i <= error_locator_poly.getdegree();i++){
        GFNUM2m coeff = error_locator_poly[i];
        if(i%2 != 0){
            dx_coeff.setc(i-1,coeff);
            // cout << "dx_coeff[" << i - 1 << "] = " << coeff <<
               endl;;
        } else {
            dx_coeff.setc(i-1, GFNUM2m(0));
            // cout << "dx_coeff[" << i - 1 << "] = 0"  << endl;
        }
        //dx_coeffs[i] = error_locator_poly[i+1]*GFNUM2m(i+1);
    }

    for(i = 0; i < n; i++){
        dec[i] = r[i];
    }
    for(i = 0; i < nroots; i++){
        errloc = (GFNUM2m::getN() - rs[i].getp()) % GFNUM2m::getN();
        GFNUM2m errval = -(Omega.evaluate(rs[i]) / (dx_coeff.
           evaluate(rs[i])));
        // cout << errloc << " " << errval << endl;
        dec[errloc] += errval;
    }

   return 1;
}


/*
Local Variables:
compile-command: "g++ -c -g RSdec.cc"
End:
*/
```

Listing 2: RSdec.cc

```
//  Program: rsencode.cc --- encode a data file
//  Todd K. Moon
//
// rsencode [-t t] infile outfile
//
// [-t t] -- specify random error correction capability.
//           default = 3

// The data are read with the first byte in the block
// corresponding to the constant coefficient of m(x).
// In order to handle the last block of the file correctly,
// the last block is written out with its length.

#include "RSenc.h"
extern "C" {
```

```
#include <stdio.h>              // fopen, fread, fwrite functions
#include <string.h>             // strcmp function
#include <stdlib.h>             // atoi function
}

int main(int argc, char *argv[])
{
   GFNUM2m::initgf(8,0x11D);  //   1 0001 1101   x^8 + x^4 + x^3 + x^2 + 1
   GFNUM2m::setouttype(vector);
   int i;
   int t = 3;                   // default error correction capability
   FILE *fin, *fout;
   if(argc < 3) {
      cout << "Usage: " << argv[0] << "[-t t] infile outfile\n";
      exit(-1);
   }
   i = 1;
   if(!strcmp(argv[i],"-t")) { // if t is specified
      ++i;
      t = atoi(argv[i++]);
   }
   int n = 255;                 // coded block length
   int k = 255-2*t;             // input block length
   RSenc encoder(n,k,t);        // build the encoder object
   unsigned char m[k];          // data space to read in
   unsigned char  c[n];         // encoded data

   if((fin = fopen(argv[i++],"rb")) ==NULL) {// open input file to read
      cout << "Cannot open file to read\n";
      exit(-1);
   }
   if((fout = fopen(argv[i],"wb")) == NULL) {  // open output file to write
      cout << "Cannot open file to write\n";
      exit(-1);
   }
   int nread, nwritten;
    GFNUM2m m_GF[k];
   while((nread = fread(m, 1, k, fin))) {
      if(nread < k) {  // last block -- not all all k bytes read
         cout << "last block: nread="<< nread << endl;
         m[nread] = nread;
         m_GF[nread] = GFNUM2m(nread);
         for(i = nread+1; i < k; i++) {
            m[i] = 0;           // zero out the rest of the block
            m_GF[i] = GFNUM2m(0);
         }
      }
```

```
      encoder.encode(m_GF);     // encode the data
      polynomialT<GFNUM2m> r = encoder.get_code();
      for(i = 0; i < n; i++){
          if(i <= r.getdegree()){
                c[i] = (unsigned char)r[i].getv();
          }else {
                c[i] = 0;
          }
      }
      nwritten = fwrite((void *)c,1,n,fout);
   }
   fclose(fin);
   fclose(fout);
}

/*
Local Variables:
compile-command: "g++ -o rsencode -g rsencode.cc RSenc.cc GFNUM2m.cc
    "
End:
*/
```

Listing 3: RSencode.cc

```
// RSenc.cc -- a general RS encoder
// Todd K. Moon
// Copyright 2004 by Todd K. Moon
// Permission is granted to use this program/data
// for educational/research only

#include "RSenc.h"
#include "polynomialT.cc"

template class polynomialT<GFNUM2m>;
template class polytemp<GFNUM2m>;

RSenc::RSenc(int n_in, int k_in, int t_in, int j0_in, GFNUM2m A1_in)
// constructor
{
   n = n_in;
   k = k_in;
   t = t_in;
   j0 = j0_in;
   A1 = A1_in;
    c.setc(n-1, GFNUM2m(0));
   g = GFNUM2m(1);
   GFNUM2m lc[2] = {1,1};
   polynomialT<GFNUM2m> l(1,lc);  // set up the linear term
   for(int i=0; i < 2*t; i++) {
      l[0] = (A1^(j0+i));
      g *= l;
```

```
28     }
29     m.setc(k-1,1);                // allocate space for message
           polynomial
30 }
31
32 GFNUM2m *
33 RSenc::encode(GFNUM2m *m_in)
34 // encode from Galios field elements
35 {
36     // fill in the blanks ...
37      polynomialT<GFNUM2m> m_poly(k, m_in);
38      polynomialT<GFNUM2m> xl(GFNUM2m(0));
39      xl.setc(n-k, GFNUM2m(1));
40      polynomialT<GFNUM2m> temp;
41      polynomialT<GFNUM2m> remainder;
42      temp = m_poly * xl;
43      temp.divide(g);
44      temp = temp + temp.getlastremainder();
45      GFNUM2m c_row[n];
46      for(int i = 0; i < n; i++){
47          if(i <= temp.getdegree())
48              c.setc(i,temp[i]);
49          else
50              c.setc(i,GFNUM2m(0));
51      }
52      GFNUM2m *dumb;
53      return dumb;
54 }
55
56 void
57 RSenc::encode(unsigned char *m_in, unsigned char *c_out)
58 // encode from chars
59 {
60     // fill in the blanks
61 }
62
63 /*
64 Local Variables:
65 compile-command: "g++ -c -g RSenc.cc"
66 End:
67 */
```

Listing 4: RSenc.cc